Programmieraufgabe: Simulation «Wolf, Schaf und Gras»

Ausbildungsprojekt · C# · WPF · Client-Server-Architektur

1. Ausgangslage

Du entwickelst eine Simulation eines vereinfachten Ökosystems. Auf einem quadratischen Raster leben Wölfe und Schafe, während Gras als Nahrungsquelle für die Schafe nachwächst.

Die Populationen beeinflussen sich gegenseitig: Schafe fressen Gras und vermehren sich dadurch. Wölfe jagen Schafe und können sich vermehren, wenn sie genügend Nahrung finden. Fehlt die Nahrung, sterben die Tiere mit der Zeit.

Deine Aufgabe ist es, dieses Ökosystem als Desktop-Anwendung mit einer grafischen Benutzeroberfläche in WPF umzusetzen. Die Anwendung muss als Client-Server-Applikation aufgebaut sein.

Du planst die Architektur, definierst die benötigten Klassen und Entitäten und implementierst anschliessend die Simulation so, dass du unterschiedliche Szenarien ausprobieren und deren Auswirkungen beobachten kannst.

2. Lernziele

Mit dieser Aufgabe vertiefst du folgende Kompetenzen:

- Du entwickelst eine Desktop-Anwendung mit C# und WPF.
- Du konzipierst eine Client-Server-Architektur und begründest die Aufteilung der Verantwortlichkeiten.
- Du modellierst ein Problem mithilfe von Klassen, Entitäten und Beziehungen.
- Du implementierst eine zeitgesteuerte Simulation mit nachvollziehbaren Spielregeln.
- Du verwendest Schieberegler, Schaltflächen und weitere GUI-Elemente zur Steuerung einer Anwendung.
- Du kommunizierst zwischen Client und Server über eine geeignete Schnittstelle.
- Du trennst Geschäftslogik, Datenhaltung und Benutzeroberfläche.
- Du testest deine Anwendung und untersuchst, wie sich veränderte Parameter auf das Ökosystem auswirken.

3. Funktionale Anforderungen

3.1 Simulationswelt

Die Welt besteht aus einem 25 × 25 Raster mit insgesamt 625 Feldern.

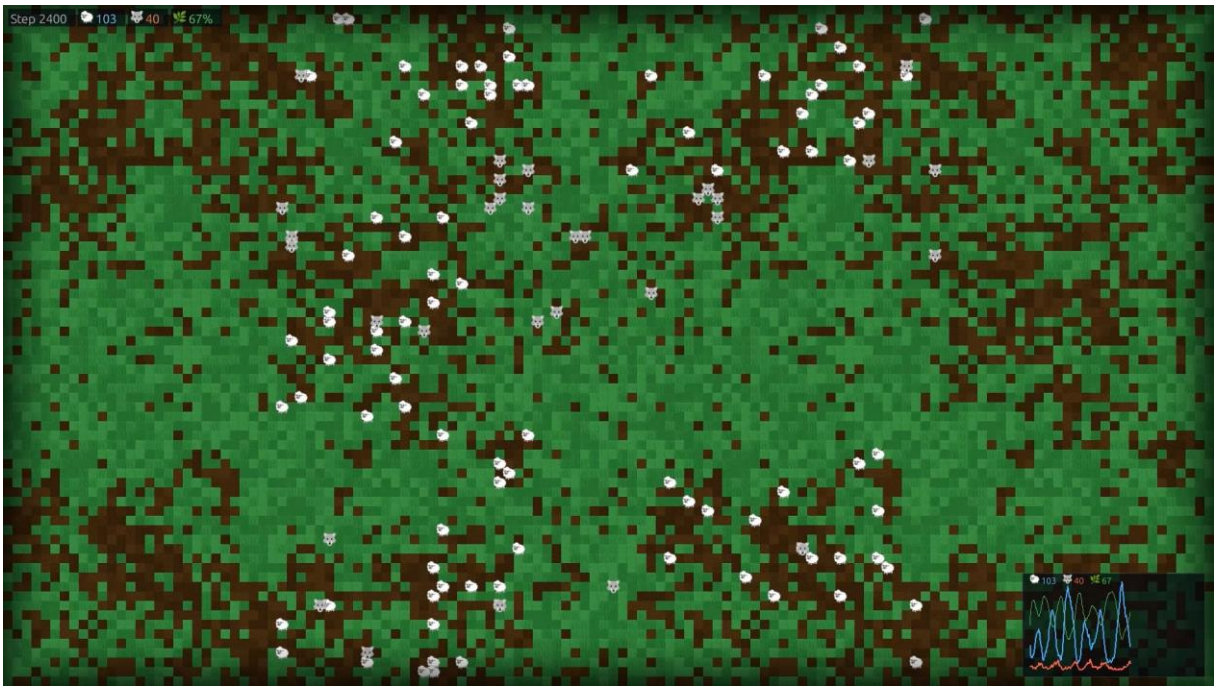
Gras -> Nahrung für Schafe

Schaf -> Frisst Gras

Wolf -> Frisst Schafe

Graswachstum -> Kurz oder lang

Die «Welt» kann ganz einfach aussehen, so zum Beispiel:



Für die Simulation gelten folgende Regeln:

- Jedes Feld kann Gras enthalten oder von einem oder mehreren Tieren belegt sein. Ob mehrere Tiere gleichzeitig auf einem Feld stehen dürfen, legst du im Rahmen der Architekturplanung fest.
- Gras wächst im Laufe der Zeit nach.
- Gras kann kurz oder lang sein. Langes Gras liefert Schafen mehr Nährwerte als kurzes Gras.
- Schafe benötigen Gras als Nahrung.
- Wölfe benötigen Schafe als Nahrung.
- Tiere können sich bewegen, Nahrung aufnehmen, sich vermehren und sterben.
- Die Populationen und der Zustand der Welt verändern sich während der Simulation laufend.

3.2 Verhalten der Tiere

Schafe

- Schafe bewegen sich durch die Welt.
- Sie können Gras fressen und dadurch Energie aufnehmen.
- Bei ausreichender Energie beziehungsweise nach Erfüllung der definierten Fortpflanzungsbedingungen können sie sich vermehren.
- Wenn ihre Energie aufgebraucht ist, sterben sie.

Wölfe

- Wölfe bewegen sich durch die Welt.
- Sie können Schafe fressen und dadurch Energie aufnehmen.

- Wenn sie genügend Nahrung aufgenommen haben und die Fortpflanzungsbedingungen erfüllt sind, können sie sich vermehren.
- Wenn sie längere Zeit keine Nahrung finden, sterben sie.

Gras

- Gras wächst automatisch nach.
- Du kannst die Wachstumsgeschwindigkeit über die Benutzeroberfläche einstellen.
- Gras besitzt mindestens zwei Wachstumsstadien: kurz und lang.
- Die beiden Stadien liefern unterschiedliche Nährwerte.

Die genauen Regeln für Energie, Hunger, Fortpflanzung und Tod definierst du vor der Implementierung. Du begründest deine Entscheidungen und dokumentierst sie.

3.3 Bewegungsverhalten

Du implementierst zwei umschaltbare Bewegungsmodi:

- Zufällige Bewegung: Tiere wählen ihr nächstes Bewegungsziel zufällig aus.
- Zielgerichtete Bewegung: Tiere versuchen, ein geeignetes Ziel zu erreichen. Schafe können beispielsweise Gras suchen, während Wölfe Schafe verfolgen.

Du entscheidest, wie ein Ziel ausgewählt wird, wie Tiere ihren Weg dorthin finden und was passiert, wenn kein geeignetes Ziel verfügbar ist.

Die Modi müssen während der Simulation über die GUI ein- und ausgeschaltet beziehungsweise gewechselt werden können.

3.4 Steuerung der Simulation

Die Benutzeroberfläche muss mindestens folgende Funktionen anbieten:

Steuerelement	**Funktion**
Start	Simulation starten
Pause	Simulation vorübergehend anhalten
Zurücksetzen	Welt neu initialisieren
Schieberegler Graswachstum	Geschwindigkeit des Grasnachwachsens einstellen
Schieberegler Hunger/Tod	Geschwindigkeit einstellen, mit der Wölfe ohne Nahrung sterben
Schieberegler Schafvermehrung	Fortpflanzungsrate der Schafe einstellen
Bewegungsmodus	Zufällige oder zielgerichtete Bewegung auswählen
Geschwindigkeitssteuerung	Geschwindigkeit der Simulation einstellen
Populationsanzeige	Anzahl der Wölfe, Schafe und Grasfelder anzeigen

Die aktuellen Werte der Schieberegler müssen sichtbar sein. Änderungen an den Einstellungen sollen entsprechend deiner Architektur während der Simulation übernommen werden können.

Zusätzlich kannst du weitere Parameter vorsehen, beispielsweise die Fortpflanzungsrate der Wölfe, die Anfangspopulationen oder die Nährwerte der Grasstadien.

4. Client-Server-Architektur

Du setzt die Anwendung als Client-Server-Applikation um. Die Aufteilung der Aufgaben zwischen Client und Server ist ein zentraler Bestandteil des Projekts.

Vor der Implementierung erarbeitest du einen Architekturvorschlag und begründest ihn.

4.1 Deine Architekturüberlegungen

Du beantwortest mindestens folgende Fragen:

1. Welche Aufgaben übernimmt der Client?
2. Welche Aufgaben übernimmt der Server?
3. Wo befindet sich die eigentliche Simulationslogik?
4. Wo werden der aktuelle Zustand der Welt und die Populationen verwaltet?
5. Wie kommunizieren Client und Server miteinander?
6. Wie werden Einstellungen vom Client an den Server übermittelt?
7. Wie erhält der Client aktualisierte Simulationsdaten?
8. Was passiert, wenn der Server nicht erreichbar ist oder die Verbindung unterbrochen wird?

Du kannst beispielsweise folgende Aufteilung untersuchen:

WPF-Client

Rasterdarstellung, Einstellungen, Steuerung, Statistiken und Benutzereingaben

HTTP / REST API mit JSON

Simulationsserver

Simulationsschritte, Bewegungen, Nahrungsaufnahme, Fortpflanzung, Tod und Weltzustand

Diese Aufteilung ist ein möglicher Vorschlag, keine vorgegebene Lösung. Du begründest, weshalb deine gewählte Architektur geeignet ist.

Wichtig: Die Simulation muss tatsächlich auf dem Server ausgeführt werden. Der Client darf die Welt nicht unabhängig simulieren und lediglich das Ergebnis anzeigen. Andernfalls wäre die Client-Server-Trennung nicht sinnvoll umgesetzt.

4.2 Kommunikation

Du wählst eine geeignete Kommunikationslösung und dokumentierst deine Entscheidung.

Mögliche Ansätze sind:

- REST API mit HTTP und JSON
- SignalR für eine fortlaufende Kommunikation zwischen Client und Server

- Eine andere begründete Lösung

Du definierst, welche Operationen der Client benötigt. Denkbar sind beispielsweise:

- Simulation starten, pausieren und zurücksetzen
- Simulationsparameter aktualisieren
- Aktuellen Zustand der Welt abrufen
- Populationsstatistiken abrufen
- Fehler und Verbindungsprobleme behandeln

Du entscheidest auch, ob der Client den Weltzustand regelmässig abfragt oder ob der Server Änderungen aktiv übermittelt.

5. Klassen- und Entitätenmodell

Du entwirfst ein Klassenmodell, das die Simulation übersichtlich abbildet. Die folgenden Klassen sind mögliche Ansatzpunkte, aber keine abschliessende Vorgabe.

Klasse / Entität	**Mögliche Verantwortung**
World	Verwaltet das Raster und den Zustand der Welt
Cell	Beschreibt ein einzelnes Rasterfeld
Animal	Gemeinsame Eigenschaften und Verhalten von Tieren
Sheep	Verhalten und Fortpflanzung der Schafe
Wolf	Jagdverhalten und Fortpflanzung der Wölfe
Grass	Wachstumsstadium und Nährwert des Grases
SimulationEngine	Führt die Simulationsschritte aus
SimulationSettings	Verwaltet die einstellbaren Parameter
SimulationStatistics	Erfasst Populationszahlen und weitere Kennzahlen
SimulationController	Steuert Start, Pause und Zurücksetzen
SimulationService	Stellt die Simulationsfunktionen über den Server bereit

Du entscheidest, welche Klassen du tatsächlich benötigst, welche Beziehungen zwischen ihnen bestehen und welche Verantwortlichkeiten du zusammenfasst oder trennst.

Dabei klärst du insbesondere:

- Welche gemeinsamen Eigenschaften besitzen Wölfe und Schafe?
- Welche Eigenschaften und Methoden sind spezifisch für die jeweilige Tierart?
- Wie modellierst du die Position eines Tieres im Raster?
- Wie repräsentierst du den Zustand eines Feldes?

- Wie verhinderst du, dass Tiere bei der Bewegung oder Fortpflanzung ungültige Zustände erzeugen?
- Wie trennst du die Simulationslogik von der WPF-Oberfläche und der Kommunikation?

6. Statistiken und Auswertung (als Zusatzaufgabe)

Die GUI soll nicht nur das Raster darstellen, sondern auch die Entwicklung des Ökosystems nachvollziehbar machen.

Du zeigst mindestens:

- Aktuelle Anzahl der Wölfe
- Aktuelle Anzahl der Schafe
- Anzahl der bewachsenen Felder
- Anzahl der Felder mit kurzem und langem Gras
- Aktueller Simulationszustand: läuft, pausiert oder beendet

Zusätzlich entwickelst du eine grafische Darstellung der Populationsentwicklung über die Zeit, beispielsweise mit einem Liniendiagramm für Wölfe und Schafe.

Du untersuchst anhand verschiedener Einstellungen, ob sich stabile Populationen entwickeln, einzelne Arten aussterben oder das gesamte Ökosystem zusammenbricht.

7. Qualitätsanforderungen

Du berücksichtigst folgende Anforderungen bei der Umsetzung:

- Du trennst Präsentation, Geschäftslogik und Kommunikation sauber voneinander.
- Du vermeidest blockierende Operationen in der WPF-Oberfläche.
- Du stellst sicher, dass der Simulationszustand konsistent bleibt.
- Du behandelst ungültige Eingaben und Kommunikationsfehler.
- Du strukturierst deinen Code verständlich und wartbar.
- Du dokumentierst wichtige Architektur- und Modellierungsentscheidungen.
- Du implementierst automatisierte Tests für zentrale Regeln der Simulation.
- Du sorgst dafür, dass die Simulation bei gleicher Ausgangslage und gleichem Zufalls-Seed reproduzierbare Ergebnisse liefern kann, sofern du dich für einen solchen Mechanismus entscheidest.

Wenn etwas nicht ganz klar ist, selber Annahmen treffen / eigene Ideen umsetzen.

Viel Spass!